

User Management System

Backend & Frontend Assignment — Project Report

Node.js · Express.js · PostgreSQL · React
MERN-Style User Management REST API

1. Project Objective

This project implements a User Management System to demonstrate the fundamentals of backend development using Node.js, Express.js, and a relational database (PostgreSQL). It exposes a set of REST APIs that allow a client to register a user, authenticate via login, fetch all users, and update or delete a user's own profile. A basic React frontend connects to these APIs so that data can be submitted and viewed end-to-end, from form input through to persistence in the database.

2. Technologies Used

Layer	Technology
Backend Runtime	Node.js
Backend Framework	Express.js
Database	PostgreSQL
Database Driver	pg (node-postgres)
Frontend	React (Vite)
Routing	React Router DOM
State Management	React Context API
API Testing	Postman / Thunder Client
Environment Config	dotenv
Cross-Origin Support	cors

3. Database Schema

A single **users** table was designed to store registration data, with constraints enforced at the database level so invalid data is rejected before persistence.

Field	Data Type	Constraints
id	SERIAL	Primary Key
reg_id	CHAR(10)	UNIQUE, NOT NULL — exactly 10 characters
fullname	VARCHAR(50)	UNIQUE, NOT NULL
email	VARCHAR(255)	UNIQUE, NOT NULL
password	VARCHAR(300)	NOT NULL, CHECK length > 8
age	INTEGER	NOT NULL, CHECK between 16 and 65
created_at	TIMESTAMP WITH TIME ZONE	DEFAULT CURRENT_TIMESTAMP

Justification for design choices:

- `reg_id` is fixed at `CHAR(10)` since registration numbers follow a consistent institutional format, and is `UNIQUE` since it identifies a student.
- `email` is `UNIQUE` and `NOT NULL` to prevent duplicate accounts and guarantee a usable login identifier.
- `password` has a minimum-length `CHECK` constraint (`>8` characters) as a baseline safeguard against weak credentials.
- `age` is constrained between 16 and 65 to keep entries within a realistic and expected range for the system's users.
- `created_at` defaults to the current timestamp automatically, so record creation time doesn't need to be supplied by the client.

4. API Endpoints

Method	Endpoint	Purpose
GET	/	Returns a welcome message confirming the server is running
GET	/users	Fetches all registered users as an array of user objects
POST	/users	Registers a new user; requires <code>fullname</code> , <code>reg_id</code> , <code>email</code> , <code>password</code> , <code>age</code>
POST	/login	Verifies email + password and returns the matched user's details
PATCH	/profile	Updates the logged-in user's own email, password, and/or age
DELETE	/profile	Deletes the logged-in user's own account, identified by <code>reg_id</code>

Example response format (POST /users):

```
{
  "status": "Success",
  "message": "Created user successfully",
  "data": {
    "id": 1,
    "fullname": "Jane Doe",
    "reg_id": "REG0000001",
    "email": "jane@example.com",
    "age": 21
  }
}
```

5. Screenshots

The following screenshots, taken in Thunder Client, show each API endpoint being exercised against the live server, including both a success and an error case.

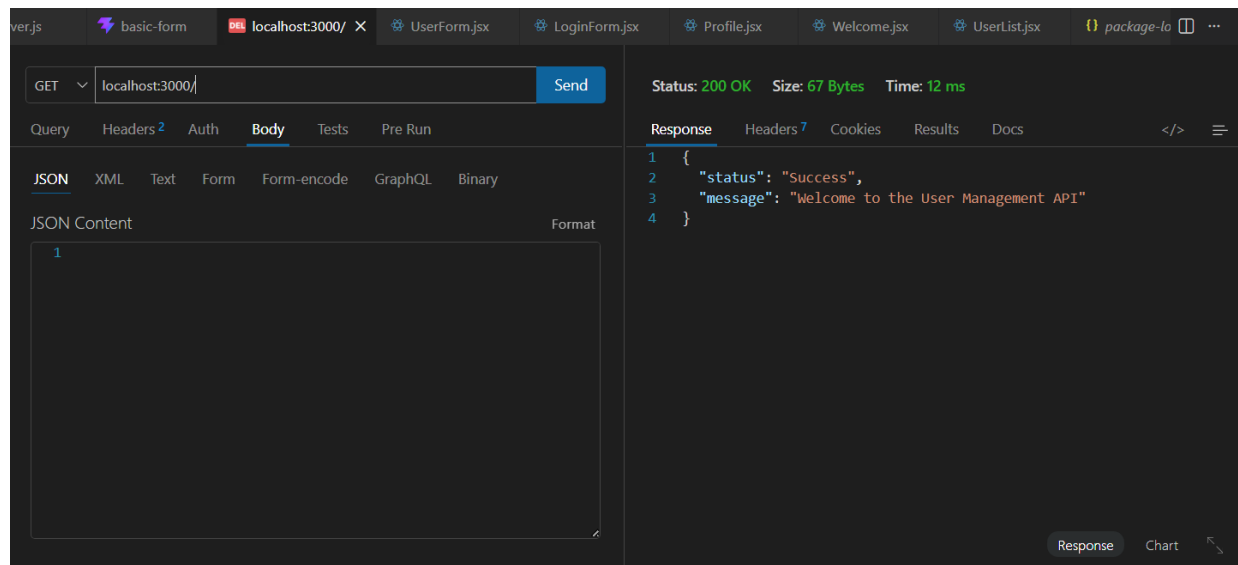


Fig 1. GET / — returns 200 OK with a welcome message confirming the server is running.

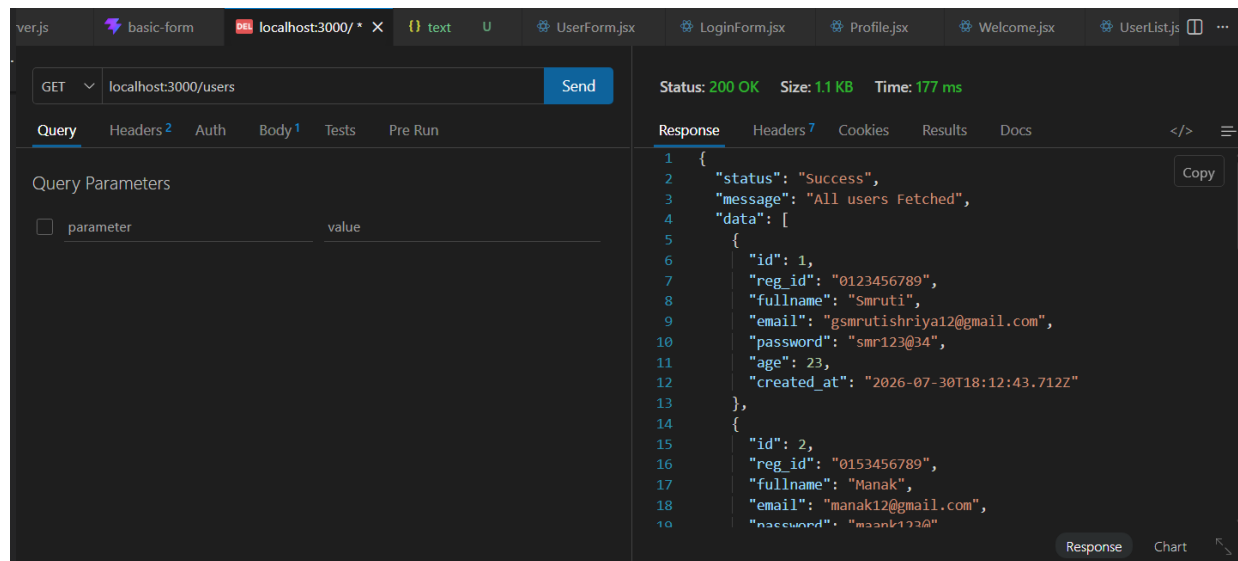


Fig 2. GET /users — returns 200 OK with an array of all registered user objects.

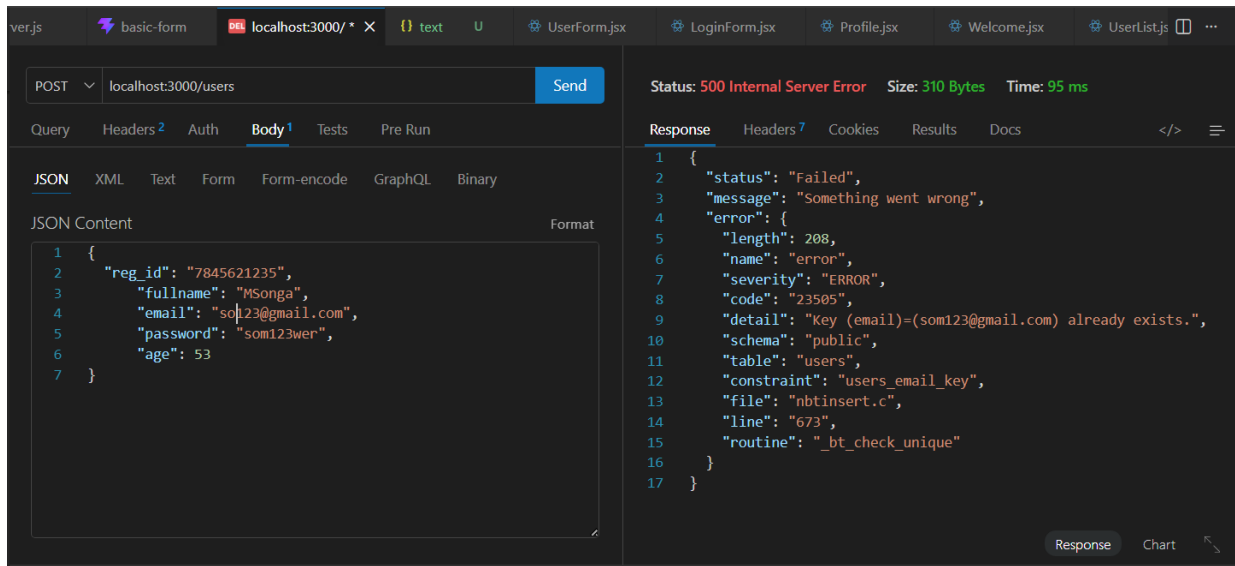


Fig 3. POST /users (error case) — returns 500 when a duplicate email is submitted, surfaced by the users_email_key unique constraint.

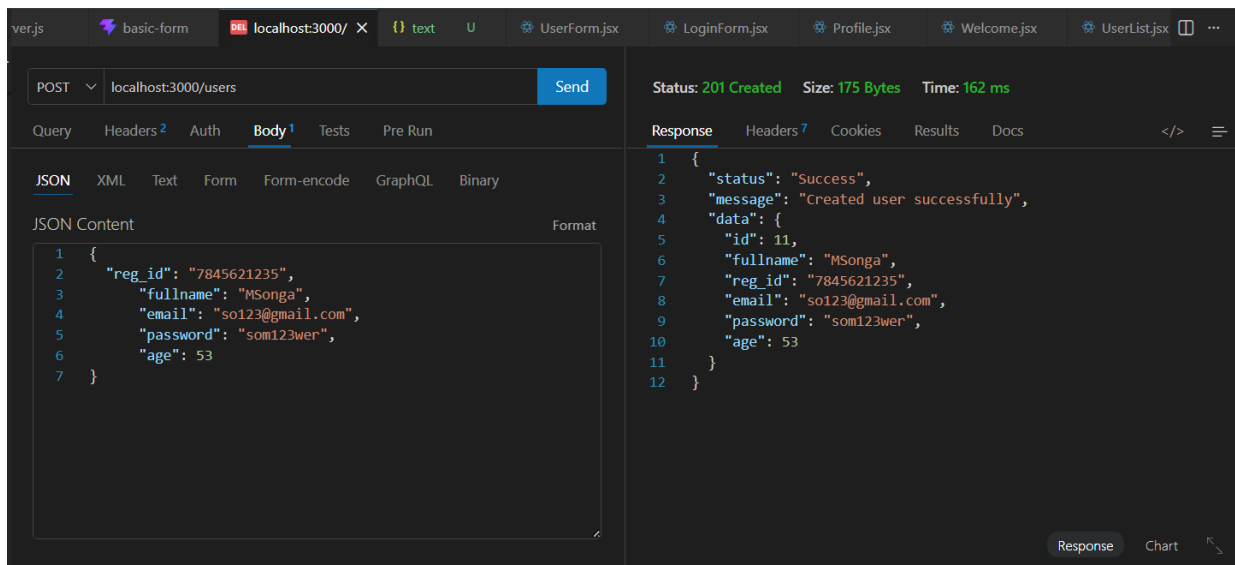


Fig 4. POST /users (success case) — returns 201 Created with the new user's data after changing to a unique email.

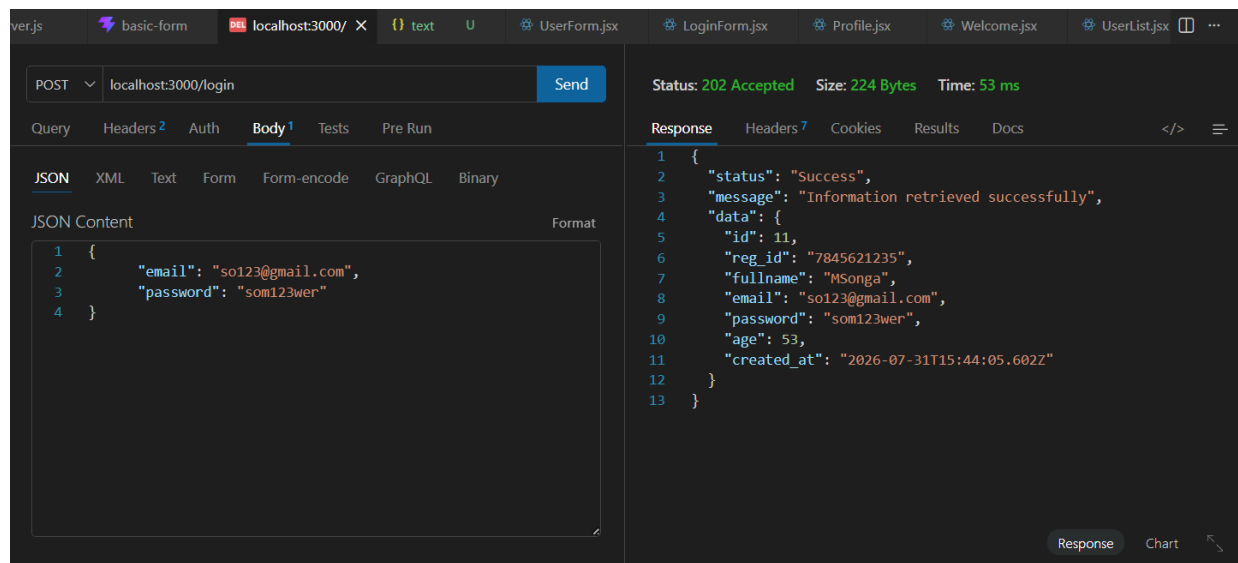


Fig 5. POST /login — returns 202 Accepted with the matched user's details for valid credentials.

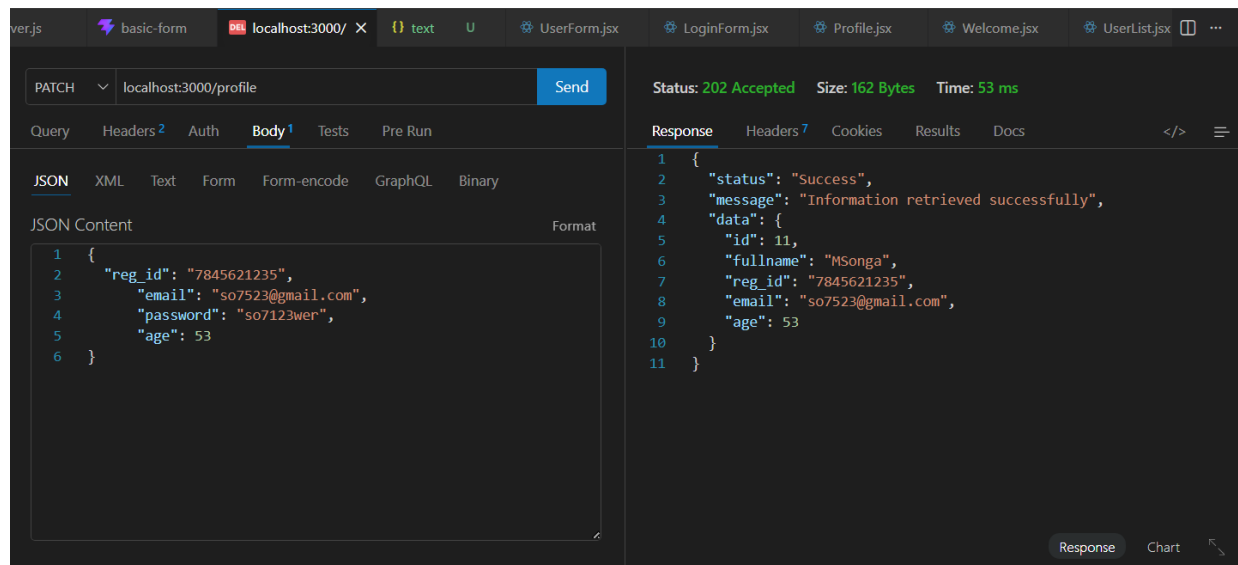


Fig 6. PATCH /profile — returns 202 Accepted with the updated email after a profile edit.

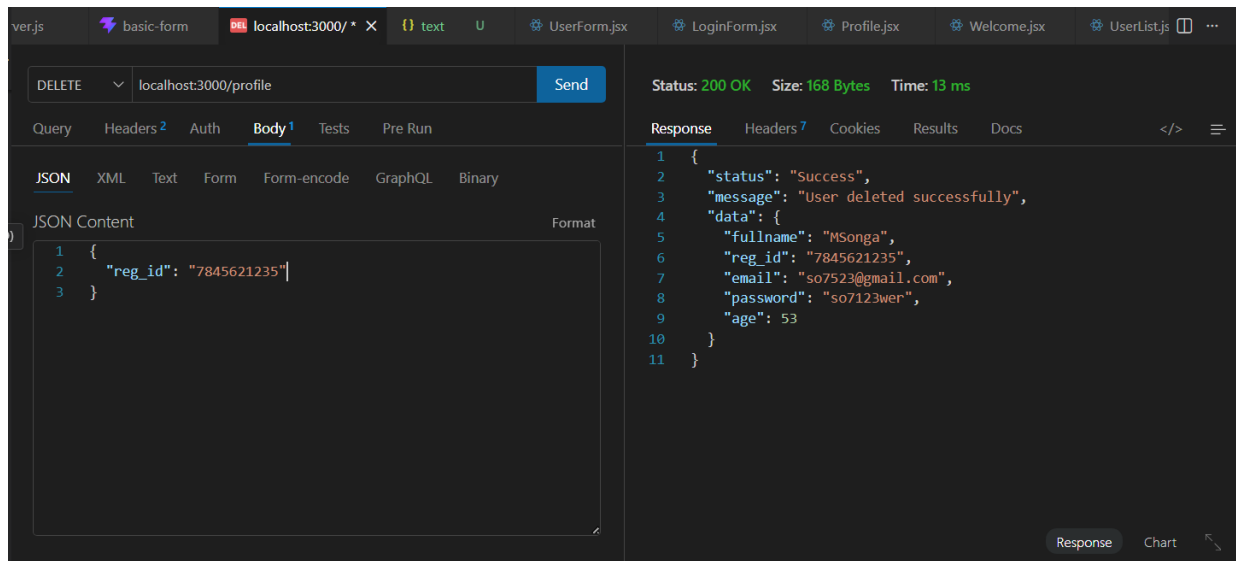


Fig 7. DELETE /profile — returns 200 OK confirming the account was deleted, along with the deleted record's data.

6. Working Explanation

The backend is an Express.js server that connects to PostgreSQL through a connection pool (`pg.Pool`). On startup, `initDatabase()` runs a `CREATE TABLE IF NOT EXISTS` query to ensure the users table exists before any routes are hit. Each route follows the same pattern: validate the incoming request body, run a parameterized SQL query against the database (to prevent SQL injection), and return a consistent JSON response shape of `{ status, message, data }`.

On the frontend, each form is a self-contained React component using `useState` to track input values and a `handleSubmit` function that calls the corresponding backend route with `fetch`. CORS is enabled on the server so the React dev server (on a different port) can make cross-origin requests. React Router DOM ties the Welcome, Register, Login, Profile, and Users List pages together, and React Context shares the logged-in user's data between the Login and Profile components after a successful login.

7. Challenges Faced

- The API appeared to read from an old/incorrect database — traced to a typo in `.env` (`PGNAME` instead of the variable `pg` actually reads, `PGDATABASE`), causing the pool to silently fall back to a default database.
- `POST /users` failed with a Postgres 'undefined column' error because the `RETURNING` clause referenced a `username` column that doesn't exist in the schema (the actual column is `fullname`).
- `DELETE /profile` returned Express's default 404 HTML page because the route was defined as `/users` instead of `/profile`, so no route matched the request.
- The frontend crashed with a Vite parse error ('await is only allowed within async functions') because `handleSubmit` used `await` without being declared `async`.

- An import error ('Failed to resolve import ../components/UserForm') occurred due to a filename case mismatch between the import path and the actual file on disk.

8. Learning Outcomes

- Environment variable names must exactly match what the underlying library expects (e.g. pg reads PGDATABASE, not arbitrary names) — a mismatch fails silently rather than throwing an error.
- SQL errors like 'undefined column' are best debugged by checking the actual table schema against every column referenced in a query, including RETURNING clauses.
- Express matches routes on both path and HTTP method exactly; a mismatch on either results in a default 404 response, not a custom error.
- Any function using await must be declared async, and this includes React event handlers like handleSubmit.
- Cross-origin requests between a frontend and backend on different ports require CORS to be explicitly enabled on the server.
- File imports are case-sensitive in practice even on case-insensitive filesystems like Windows, once bundlers like Vite are involved.

9. AI Usage Report

AI (Claude, Anthropic) was used strictly for debugging and understanding frontend-backend communication, not for generating backend or frontend implementation code. Full details, including prompts used and issues resolved, are documented separately in ProjectReport.md / AI_Log.md as required by the assignment's AI usage policy.

- Explained general patterns for fetching an array of user objects in Node.js (used as a reference to confirm existing route logic).
- Diagnosed the .env PGNAME/PGDATABASE mismatch causing the app to connect to the wrong database.
- Diagnosed the Postgres 42703 'undefined column' error from a mismatched RETURNING clause.
- Explained how to enable CORS and connect a React form to an Express API using fetch.
- Diagnosed a route mismatch (/users vs /profile) causing a 404 on DELETE.
- Diagnosed a missing async keyword causing a Vite parse error.
- Diagnosed a case-sensitive import path causing a module resolution failure.

10. Error Summary

#	Error	Cause	Fix
---	-------	-------	-----

1	Wrong database connected	.env used PGNAME instead of PGDATABASE	Renamed to PGDATABASE
2	42703: undefined column	RETURNING referenced username, which doesn't exist	Changed to fullname
3	Cannot DELETE /profile	Route was defined as /users, not /profile	Renamed route to /profile
4	await outside async	handleSubmit missing the async keyword	Added async to handleSubmit
5	Failed to resolve import	Filename case mismatch (Userform.jsx vs UserForm.jsx)	Renamed file to match import
6	500 on duplicate email	Email already exists (users_email_key unique constraint)	Used a unique email on retry